10-152-117 Python Programming Foundations

Course Outcome Summary

COURSE INFORMATION

Description:
Learners explore Python programming by designing and implementing solutions to real-world problems. Students develop foundational programming skills using variables, data types, conditionals, loops, functions, and data structures. The course emphasizes problem-solving, debugging, and code comprehension while introducing the use of AI-assisted tools to support development. Learners practice building, evaluating, and refining programs, both independently and with AI assistance. By the end of the course, students are able to create, understand, and explain basic Python programs and apply structured thinking to solve problems.

Career Cluster: Information Technology
Instructional Level: A.A.S. - Associate in Applied Science
Total Credits: 2
Total Hours: 54

TEXTBOOKS

Learn Python Programming: A comprehensive, up-to-date, and definitive guide to learning Python - 4th ed. Edition - Fabrizio Romano/Heinrich Kruger

The Missing Tools: How Developers Learn to Think Quickly - 1st Edition - Cash Myers

Python Programming Language: a QuickStudy Laminated Reference Guide Pamphlet – Robin Nixon – (Optional)

CORE ABILITIES

1. **Act Professionally**

   Status: Active

   Criteria

   1.1. Take responsibility for one's own actions.
   1.2. Conform to the technical standards of a profession.
   1.3. Practice morally responsible behavior.
   1.4. Take responsibility for staying up-to-date
   1.5. Maintain confidentiality.
   1.6. Exhibit respect for people and property.
   1.7. Exhibit a sense of global awareness.
   1.8. Display appearance appropriate for work environment.

2. **Communicate Clearly**

Status: Active

Criteria

2.1. Speak clearly so others can understand.
2.2. Write with clarity so that others can understand.
2.3. Select appropriate means to convey a message.
2.4. Ask questions for clarification.
2.5. Participate in discussions and group presentations.
2.6. Interpret nonverbal communications.
2.7. Use active listening skills.
2.8. Apply standards of spelling, English, grammar, and punctuation.

3. Solve Problems

Status: Active

Criteria

3.1. Use critical, creative thinking skills.
3.2. Apply problem solving steps.
3.3. Demonstrate open-mindedness.
3.4. Evaluate alternatives to choosing a solution.
3.5. Implement solutions appropriately.
3.6. Use research to solve problems.
3.7. Use appropriate mathematical calculations.

4. Value Learning

Status: Active

Criteria

4.1. Assume responsibility for lifelong learning.
4.2. Identify own learning needs.
4.3. Apply effective learning processes.
4.4. Help other to learn effectively.

5. Work Cooperatively

Status: Active

Criteria

5.1. Demonstrate respect for self and others.
5.2. Contribute to a group activity with ideas, suggestions, and effort.
5.3. Complete own share of tasks necessary to finish a group project.
5.4. Maintain a safe and healthy work environment for self/group.
5.5. Set goals, standards, limits, for self/group.
5.6. Display effective interpersonal skills.
5.7. Resolve conflicts in a constructive manner.
5.8. Seek help when needed.
5.9. Demonstrate ability to work with a diverse population.

6. Work Productively

Status: Active

Criteria

6.1. Attend regularly and on time.
6.2. Exhibit organizational skills.
6.3. Locate resources for problem solving.
6.4. Display productive work ethic.
6.5. Maintain necessary knowledge and skills.
6.6. Use effective, efficient processes.
6.7. Show self-direction in starting tasks.
6.8. Demonstrate reliability.
6.9. Follow directions.
6.10. Complete required tasks on time.

COURSE COMPETENCIES

1. Apply Python fundamentals to construct working programs

Domain: Cognitive Level: Applying Status: Active

Assessment Strategies

1.1. Lab-based programming assignments
1.2. Project assignments

Criteria

1.1. Write Python statements using correct syntax
1.2. Use variables, values, expressions, and basic data types appropriately
1.3. Apply input and output operations in simple programs
1.4. Interpret and correct basic syntax or type-related issues
1.5. Produce readable code that runs as intended

Learning Objectives

1.a. Identify Python syntax, values, variables, and expressions
1.b. Explain how basic data types are used in Python programs
1.c. Describe how input and output support program interaction

2. Design program logic using control flow and sequencing

Domain: Cognitive Level: Analyzing Status: Active

Assessment Strategies

2.1. Lab-based programming assignments
2.2. Project assignments
2.3. Logic tracing activities

Criteria

2.1. Break problems into step-by-step program logic
2.2. Use conditionals, loops, and Boolean logic appropriately
2.3. Sequence instructions so program behavior matches the task
2.4. Identify inputs, outputs, constraints, and expected behavior

2.5. Trace program flow across expected cases

Learning Objectives

2.a. Define program logic and control flow

2.b. Explain the role of conditionals, loops, and sequencing

2.c. Describe how problem requirements become program behavior

3. Organize Python code using functions, collections, and readable structure

Domain: Cognitive Level: Applying Status: Active

Assessment Strategies

3.1. Lab-based programming assignments

3.2. Project assignments

3.3. Code review or explanation activities

Criteria

3.1. Define and use functions to organize program behavior

3.2. Use parameters, return values, and collections appropriately

3.3. Choose structures that make code easier to read and maintain

3.4. Reduce unnecessary repetition in program code

3.5. Recognize simple class-based or application structures when introduced

Learning Objectives

3.a. Explain how functions support code organization

3.b. Describe how lists, dictionaries, and other collections organize data

3.c. Identify characteristics of readable and maintainable code

4. Handle data using collections, files, structured formats, and integrations

Domain: Cognitive Level: Applying Status: Active

Assessment Strategies

4.1. Lab-based programming assignments

4.2. Project assignments

4.3. Data-focused programming tasks

Criteria

4.1. Store and retrieve data using appropriate Python structures

4.2. Read from and write to files when required

4.3. Work with structured formats such as CSV or JSON at an introductory level

4.4. Interpret data from simple external or simulated sources

4.5. Select data representations that support the program task

Learning Objectives

4.a. Identify common Python data structures and file patterns

4.b. Explain how structured data can be read, stored, and represented

4.c. Describe how data handling supports larger program behavior

5. Test, debug, and validate Python programs

Domain: Cognitive Level: Evaluating Status: Active

Assessment Strategies

5.1. Lab-based programming assignments
5.2. Project assignments
5.3. Debugging exercises
5.4. Program explanation checks

Criteria

5.1. Identify syntax, runtime, and logic errors
5.2. Use sample inputs to test program behavior
5.3. Compare expected results with actual results
5.4. Repair program issues and validate the fix
5.5. Explain why a corrected solution works

Learning Objectives

5.a. Define debugging, testing, and validation
5.b. Identify common Python programming errors
5.c. Explain how test cases support program correctness

6. Use AI/RBA-assisted development responsibly and transparently

Domain: Cognitive Level: Evaluating Status: Active

Assessment Strategies

6.1. Project assignments
6.2. AI-use justification activities
6.3. Prompting and revision exercises
6.4. Final project documentation

Criteria

6.1. Use AI tools within assignment boundaries
6.2. Frame prompts that describe intended program behavior
6.3. Evaluate AI-generated code before using it
6.4. Adapt, reject, or revise AI output based on requirements
6.5. Explain the human decisions made during AI-assisted development

Learning Objectives

6.a. Describe how AI tools can support programming work
6.b. Identify appropriate and inappropriate uses of AI assistance
6.c. Explain how responsible AI use requires validation and accountability

7. Solve programming problems using structured strategies

Domain: Cognitive Level: Analyzing Status: Active

Assessment Strategies

7.1. Lab-based programming assignments
7.2. Troubleshooting activities
7.3. Project assignments
7.4. Instructor observation

Criteria

7.1. Identify the source of a programming or workflow problem
7.2. Apply problem-solving steps before requesting direct answers
7.3. Use resources, examples, documentation, or targeted support to recover
7.4. Compare possible solutions before selecting an approach
7.5. Persist through revision when an initial solution does not work

Learning Objectives

7.a. Describe structured problem-solving strategies for programming tasks
7.b. Explain how troubleshooting supports independent recovery
7.c. Identify resources that can support technical problem solving

8. Communicate programming logic and development decisions clearly

Domain: Cognitive Level: Applying Status: Active

Assessment Strategies

8.1. Code explanation activities
8.2. Project documentation
8.3. AI-use justifications
8.4. Presentations or discussion

Criteria

8.1. Explain what their code does in clear language
8.2. Describe the logic and decisions behind a solution
8.3. Ask clarifying questions when requirements are unclear
8.4. Document AI use, sources, or design choices when required
8.5. Communicate project outcomes with appropriate technical vocabulary

Learning Objectives

8.a. Explain the role of communication in programming work
8.b. Identify ways to describe code behavior and design decisions
8.c. Describe how clear documentation supports technical accountability

9. Work productively through lab time, iteration, and assignment completion

Domain: Affective Level: Responding Status: Active

Assessment Strategies

9.1. Lab-based programming assignments
9.2. Project milestones
9.3. Instructor observation
9.4. Assignment submission patterns

Criteria

9.1. Use lab time to make visible progress on programming tasks
9.2. Organize files, notes, and assignment materials
9.3. Follow directions and assignment requirements
9.4. Complete required tasks by stated deadlines
9.5. Demonstrate self-direction when beginning or continuing work

Learning Objectives

9.a. Identify productive work habits for programming labs

9.b. Explain how organization supports assignment completion

9.c. Describe how iteration contributes to stronger technical work

10. Apply feedback, resources, and revision to improve learning

Domain: Affective Level: Valuing Status: Active

Assessment Strategies

10.1. Revised assignments

10.2. Reflection activities

10.3. Project checkpoints

10.4. Instructor observation

Criteria

10.1. Use instructor feedback to improve program quality

10.2. Locate and apply appropriate learning resources

10.3. Revise code, explanations, or project decisions based on evidence

10.4. Ask useful questions that support deeper understanding

10.5. Demonstrate growth across repeated programming tasks

Learning Objectives

10.a. Explain how feedback supports learning in programming

10.b. Identify resources that can improve technical understanding

10.c. Describe how revision strengthens both code and reasoning

11. Collaborate respectfully in programming and technical learning activities

Domain: Affective Level: Responding Status: Active

Assessment Strategies

11.1. Pair or group activities

11.2. Peer support activities

11.3. Discussion or presentation

11.4. Instructor observation

Criteria

11.1. Participate respectfully in technical learning activities

11.2. Contribute ideas, effort, or support during collaborative work

11.3. Seek help appropriately when blocked

11.4. Support peers without replacing their learning or work

11.5. Resolve confusion or disagreement constructively

Learning Objectives

11.a. Describe expectations for respectful technical collaboration

11.b. Explain how peer support can strengthen learning

11.c. Identify appropriate boundaries for collaboration in programming work

12. Act professionally in technical coursework and tool use

Domain: Affective Level: Valuing Status: Active

Assessment Strategies

12.1. Lab participation

12.2. Project assignments

12.3. AI-use documentation

12.4. Instructor observation

12.5. Final project presentation or submission

Criteria

12.1. Take responsibility for their work, conduct, and tool use

12.2. Follow course, assignment, and technical standards

12.3. Use AI, code examples, and external resources ethically

12.4. Respect people, property, data, and learning environments

12.5. Demonstrate readiness, reliability, and accountability in technical tasks

Learning Objectives

12.a. Explain professional expectations in programming coursework

12.b. Identify ethical considerations in AI-assisted technical work

12.c. Describe how accountability supports trust in software development

COURSE LEARNING PLANS AND PERFORMANCE ASSESSMENT TASKS

- LP: Learning plans can be found in the learning management system